

PROJECT SPEC - BoxdIn

GENERAL DESCRIPTION

This app is called BoxdIn, short for "Letterboxd Insights." It is designed for users of Letterboxd, a social networking app for dedicated movie fans. Users can log films in a diary, rate them, write reviews, and engage with a community of fellow film watchers. While Letterboxd's free tier allows full use of these features, most stat-tracking and viewing insights are locked behind a paid subscription. BoxdIn aims to give free users access to a meaningful set of personalized stats and insights at no cost.

Every Letterboxd user profile has a publicly accessible RSS feed. When a user opens BoxdIn and enters their Letterboxd username, the app fetches data from that feed and generates a dashboard of insights about their recent viewing habits. Some additional movie data is pulled from the OMDb API, a free web service that provides information from IMDb.

Data used from the Letterboxd RSS feed:

- Film title
- Watch date
- User star rating
- Film release year
- Rewatch flag (yes/no)

Data used from the OMDb API:

- Genre
- Director
- Runtime
- IMDb rating

BoxdIn displays insights in three sections:

- Snapshot (Your last 30 days)
 - *Number of films watched*
 - *Total cumulative hours watched*
 - *Most watched genre*
- Variety
 - *Number of unique genres watched*
 - *Number of unique directors watched*
 - *Oldest and newest film watched*
- Rating Breakdown:
 - *Average rating by genre (min. 3 films)*
 - *Average rating by director (min. 3 films)*
 - *Average rating by decade (min. 3 films)*
 - *Overall average difference between your ratings and IMDb ratings*
 - *Most controversial opinion (the film where your rating differed most from IMDb)*

For version 1, BoxdIn will run as a simple command line app. The user types in a Letterboxd username, hits enter, and the results print out in the terminal. It's not flashy, but it will let me focus on making sure the data and insights pull and populate correctly first. Once that's ready, version 2 would ideally have a proper web interface built with Flask, similar to the RSS feed project from last semester. The user would type their username into a text field on a webpage and see the results displayed in a clean, readable layout. Some additional features that would be nice to add in version 2 are a few charts and graphs. I'd like to have a bar chart that shows the number of films the user has watched from each decade, a pie chart of movie genres, a histogram of their ratings, and a chart showing most watched directors. These would work better in a web interface, so I think I'll save those for v 2.0.

TASK VIGNETTES

Vignette 1: Opening the app and entering a username

Chloe is an avid Letterboxd user who logs almost every film she watches. She's curious about her viewing habits but doesn't want to pay for a Letterboxd Pro subscription just to see some basic stats. She opens the app, and is prompted to enter her Letterboxd username. She types it in and hits enter. A moment later her insights are displayed.

Technical details:

- Chloe runs the script from the command line
- App prints a welcome message and prompts for a Letterboxd username
- App always fetches the fresh RSS feed from Letterboxd using the entered username
- The feed is parsed and the relevant data points are pulled out for each film
- App checks if a saved data file already exists for this username
- If it does, app only queries OMDb for films that aren't already in the saved file
- If no saved file exists, app queries OMDb for all 50 films
- All data is saved to a file so future lookups require fewer OMDb calls
- App confirms the data is ready and displays the insights

Vignette 2: Missing data

Chloe tends to log films quickly after watching them, but sometimes she logs a film without leaving a rating. When BoxdIn generates her insights, films without a rating are still counted toward her total films watched and hours watched, but are left out of any calculations that require a star rating. Chloe doesn't notice anything is wrong because the app just works around the gaps.

Technical details:

- When parsing each RSS entry, app checks whether a rating exists before including it in rating-based calculations
- Films with no rating are still counted for non-rating insights like total films watched, hours watched, genre counts, and director counts
- If OMDb returns incomplete data for a film (e.g. no runtime or no genre), that film is skipped for any insight that requires that field
- App does not crash or display an error, it simply works with whatever data is available

Vignette 3: Invalid username

Chloe wants to show BoxdIn to her friend and types in his Letterboxd username, but makes a typo. The app tries to fetch the RSS feed but comes back empty. Rather than crashing or displaying a confusing error message, BoxdIn lets her know that the username wasn't found and prompts her to try again.

Technical details:

- App attempts to fetch the RSS feed using the entered username
- If the feed returns empty or can't be reached, app prints a simple message like "Username not found, please try again"
- App prompts the user to enter a new username rather than closing
- No data is saved for an invalid username

TECHNICAL FLOW

Input

- User enters a Letterboxd username as a text string via the command line

Fetch and parse RSS feed

- App requests the RSS feed from `letterboxd.com/[username]/rss/`
- Feed is returned as XML and parsed using feedparser
- Relevant fields are extracted from each entry: film title, watch date, user rating, release year, rewatch flag
- Result is a list of dictionaries, one per film

Check for saved data (*you recommended this, it was a great suggestion!*)

- App checks if a CSV file already exists for this username
- Any films already in the CSV are skipped when querying OMDb
- Only new films get sent to OMDb

Fetch data from OMDb

- For each new film, app sends a request to the OMDb API using the film title
- OMDb returns JSON containing genre, director, runtime, and IMDb rating
- If any of these fields are missing, that film is noted but not discarded entirely

Save combined data

- RSS data and OMDb data are combined into a pandas dataframe
- Dataframe is saved as a CSV file named after the username

Calculate insights

- The Snapshot section filters for only films watched in the last 30 days before running its calculations
- All other sections use the full dataset of up to 50 films
- Any insight requiring a rating skips films with no rating

- Any insight requiring OMDb data skips films where that data is missing
- Rating breakdown sections skip any category with fewer than 3 films

Display results

- Results are printed to the terminal in three sections: Snapshot, Variety, and Rating Breakdown

Technical Feasibility

- The RSS feed parsing is the part I'm most confident about. We worked with RSS feeds and feedparser in HCI 574, so I know that piece works. We also built a Flask web app in that class, so the version 2 web interface is something I have some prior experience with.
- The OMDb API queries and CSV caching are things I haven't tried yet, but your example code for querying OMDb gives me a good starting point. The main unknown is how reliably OMDb returns clean data for all 50 films and how long 50 sequential requests will take to run.

SELF ASSESSMENT

The biggest change from my original sketch wasn't really a change. I had identified the API and the general concept, but I hadn't thought through what insights to actually show. Working through the spec, I ended up with a lot of ideas, and I enjoyed brainstorming different ways to give new context to the data. I ended up scrapping several ideas that weren't interesting enough or didn't fit well with the others. I'm happy with the list I settled on.

I've never done most of what this app requires, but I think it's all achievable. I can picture how it fits together in my head, which feels like a good sign. The biggest challenge will probably just be the sheer amount of work. There are a lot of individual insights to build out, and each one needs to work correctly and handle missing data properly. It's not that any single piece seems impossible, it's just that there's a lot of it.

The part I'm least confident about is working with the OMDb API. Making sure the data comes back correctly for all 50 films and gets stored and displayed properly is where I'd expect to hit a few potholes and have to iron out some kinks.